

DBMS Project query output

```
-- =====
```

```
-- 1. List all users (simple SELECT)
```

```
-- =====
```

```
SELECT id, email, plan_id, is_active  
FROM users;
```

	id	email	plan_id	is_active
▶	11111111-1111-1111-1111-111111111111	alice@example.com	3	1
	22222222-2222-2222-2222-222222222222	bob@example.com	2	1
	33333333-3333-3333-3333-333333333333	carol@example.com	1	1
•	NULL	NULL	NULL	NULL

```
-- =====
```

```
-- 2. Get all profiles of a user
```

```
-- =====
```

```
SELECT p.id, p.name, p.is_kids  
FROM profiles p;
```

	id	name	is_kids
▶	aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa1	Alice-main	0
	aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa2	Kiddo	1
	bbbbbbbbb-bbbb-bbbb-bbbb-bbbbbbbbbbbb1	Bob-main	0
	cccccccc-cccc-cccc-cccc-cccccccccccc1	Carol-main	0

```
-- =====
```

```
-- 3. List all content titles (ORDER BY)
```

```
-- =====
```

```
SELECT title, type, release_date  
FROM content  
ORDER BY release_date DESC;
```

	title	type	release_date
▶	Life of Warriors	movie	2023-04-12
	Kids Show!	series	2022-07-01
	Space Quest	series	2021-05-01
	Comedy Night	movie	2020-08-15
	History of Earth	movie	2019-01-20

```
-- =====
```

```
-- 4. Get all movies only (WHERE filter)
```

```
-- =====
```

```
SELECT id, title
```

```
FROM content
WHERE type = 'movie';
```

	id	title
▶	c2	Comedy Night
	c3	History of Earth
	c5	Life of Warriors
★	NULL	NULL

```
-- =====
-- 5. Count how many contents per type
-- =====
SELECT type, COUNT(*) AS total
FROM content
GROUP BY type;
```

	type	total
▶	series	2
	movie	3

```
-- =====
-- 6. Get all episodes of a series (JOIN)
-- =====
SELECT e.season, e.episode_no, e.title
FROM episodes e
JOIN content c ON e.content_id = c.id
WHERE c.slug = 'space-quest'
ORDER BY e.season, e.episode_no;
```

	season	episode_no	title
▶	1	1	Episode 1
	1	2	Episode 2
	1	3	Episode 3

```
-- =====
-- 7. Simple search (LIKE)
-- =====
SELECT id, title
FROM content
```

WHERE title LIKE '%Quest%';

	id	title
▶	c1	Space Quest
*	NULL	NULL

-- =====

-- 8. Get genres for a content (JOIN many-to-many)

-- =====

SELECT g.name AS genre

FROM genres g

JOIN content_genres cg ON g.id = cg.genre_id;

	genre
▶	Comedy
	Documentary
	Drama
	Drama
	Kids
	Sci-Fi

-- =====

-- 9. Profiles + their device count (LEFT JOIN + GROUP BY)

-- =====

SELECT p.name, COUNT(d.id) AS device_count

FROM profiles p

LEFT JOIN devices d ON p.id = d.profile_id

GROUP BY [p.id](#);

	name	device_count
▶	Alice-main	1
	Kiddo	0
	Bob-main	1
	Carol-main	0

```
-- =====
```

```
-- 10. Recently added content (LIMIT)
```

```
-- =====
```

```
SELECT id, title, created_at
FROM content
ORDER BY created_at DESC
LIMIT 5;
```

Result Grid



Filter Rows:

Edit: 

	id	title	created_at
▶	c1	Space Quest	2025-11-14 14:07:21
	c2	Comedy Night	2025-11-14 14:07:21
	c3	History of Earth	2025-11-14 14:07:21
	c4	Kids Show!	2025-11-14 14:07:21
	c5	Life of Warriors	2025-11-14 14:07:21
✱	NULL	NULL	NULL

```
-- =====
```

```
-- 11. Get completed watches for a profile
```

```
-- =====
```

```
SELECT c.title, w.watched_seconds
FROM watch_history w
JOIN content c ON c.id = w.content_id
WHERE w.finished = TRUE;
```

Result Grid	Filter Rows:	Export
	title	watched_seconds
▶	Comedy Night	5400
	Life of Warriors	3500

```
-- =====
```

```
-- 12. Get top 5 highest rated content globally
```

```
-- =====
```

```
SELECT c.title, AVG(r.rating) AS avg_rating
FROM ratings r
JOIN content c ON c.id = r.content_id
GROUP BY c.id
ORDER BY avg_rating DESC
LIMIT 5;
```

	title	avg_rating
▶	Space Quest	5.0000
	Comedy Night	4.0000
	History of Earth	3.0000

```
-- =====
-- 13. Correlated subquery: user's rating for each content
-- =====
```

```
SELECT c.title,
       (SELECT rating FROM ratings r WHERE r.content_id = c.id AND r.profile_id = "%aaa%" LIMIT
        1) AS my_rating
FROM content c;
```

	title	my_rating
▶	Space Quest	5
	Comedy Night	NULL
	History of Earth	NULL
	Kids Show!	NULL
	Life of Warriors	NULL

```
-- =====
-- 14. Subquery IN: get content in Sci-Fi genre
-- =====
```

```
SELECT title
FROM content
WHERE id IN (
  SELECT content_id
  FROM content_genres cg
  JOIN genres g ON cg.genre_id = g.id
  WHERE g.name = 'Sci-Fi'
);
```

Result Grid	
	title
▶	Space Quest

```
-- =====
-- 15. CTE: Continue watching list
-- =====
```

```
WITH cw AS (
```

```

SELECT w.*, ROW_NUMBER() OVER (ORDER BY w.last_watched_at DESC) AS rn
FROM watch_history w
WHERE w.profile_id = "%aaa%" AND w.finished = FALSE
)
SELECT c.title, cw.watched_seconds, cw.last_watched_at
FROM cw
JOIN content c ON c.id = cw.content_id
WHERE cw.rn <= 10;

```

	title	watched_seconds	last_watched_at
▶	Space Quest	1200	2025-11-11 14:07:21

```

-- =====
-- 16. Find content with no ratings (NOT EXISTS)
-- =====
SELECT c.id, c.title
FROM content c
WHERE NOT EXISTS (
  SELECT 1 FROM ratings r WHERE r.content_id = c.id
);

```

Result Grid		Filter Rows:	Edit:
	id	title	
▶	c4	Kids Show!	
	c5	Life of Warriors	
✱	NULL	NULL	

```

-- =====
-- 17. How many profiles each user has
-- =====
SELECT u.email, COUNT(p.id) AS profiles_count
FROM users u
LEFT JOIN profiles p ON p.user_id = u.id
GROUP BY u.id;

```

	email	profiles_count
▶	alice@example.com	2
	bob@example.com	1
	carol@example.com	1

```
-- =====
```

```
-- 18. Total watch time per profile (SUM)
```

```
-- =====
```

```
SELECT p.name, SUM(w.watched_seconds) AS total_seconds
FROM profiles p
LEFT JOIN watch_history w ON w.profile_id = p.id
GROUP BY p.id;
```

	name	total_seconds
▶	Alice-main	4700
	Kiddo	NULL
	Bob-main	5400
	Carol-main	1800

```
-- =====
```

```
-- 19. Window function: rank top content by watch count
```

```
-- =====
```

```
WITH stats AS (
  SELECT content_id, COUNT(*) AS views
  FROM watch_history
  GROUP BY content_id
)
SELECT c.title, s.views,
       RANK() OVER (ORDER BY s.views DESC) AS ranking
FROM stats s
JOIN content c ON c.id = s.content_id;
```

	title	views	ranking
►	Space Quest	1	1
	Comedy Night	1	1
	History of Earth	1	1
	Life of Warriors	1	1

```
-- =====
-- 20. Recommendation: genre overlap with a given content
-- =====
```

```
SELECT c2.id, c2.title,
(
  SELECT COUNT(*)
  FROM content_genres cg2
  WHERE cg2.content_id = c2.id
  AND cg2.genre_id IN (
    SELECT genre_id
    FROM content_genres cg
    WHERE cg.content_id = 'c3'
  )
) AS overlap_score
FROM content c2
WHERE c2.id <> 'c3'
ORDER BY overlap_score DESC
LIMIT 10;
```

	id	title	overlap_score
►	c1	Space Quest	0
	c2	Comedy Night	0
	c4	Kids Show!	0
	c5	Life of Warriors	0

```
-- =====
-- 21. Episode completion: how many episodes watched for a series
-- =====
```

```
WITH ep_total AS (
  SELECT COUNT(*) AS total_eps
```



```

FROM episodes
WHERE content_id = 'c3'
),
ep_watched AS (
  SELECT COUNT(DISTINCT episode_id) AS watched_eps
  FROM watch_history
  WHERE profile_id LIKE '%bbb%'
  AND content_id = 'c3'
)
SELECT total_eps, watched_eps
FROM ep_total, ep_watched;

```

	total_eps	watched_eps
▶	0	0

```

-- =====
-- 22. Payments in last 30 days
-- =====
SELECT u.email, p.amount_cents, p.created_at
FROM payments p
JOIN users u ON u.id = p.user_id
WHERE p.created_at >= NOW() - INTERVAL 30 DAY;

```

Result Grid Filter Rows: Export: Wrap Cell C			
	email	amount_cents	created_at
▶	alice@example.com	1299	2025-10-25 14:07:21
	bob@example.com	799	2025-11-04 14:07:21

```

-- =====
-- 23. Most active profiles (ORDER BY SUM)
-- =====
SELECT p.name, SUM(w.watched_seconds) AS total_watch
FROM profiles p
JOIN watch_history w ON w.profile_id = p.id
GROUP BY p.id
ORDER BY total_watch DESC

```

LIMIT 5;

	name	total_watch
▶	Bob-main	5400
	Alice-main	4700
	Carol-main	1800

-- =====

-- 24. Content with multiple genres (HAVING > 1)

-- =====

```
SELECT c.title, COUNT(cg.genre_id) AS genre_count
FROM content c
JOIN content_genres cg ON c.id = cg.content_id
GROUP BY c.id
HAVING genre_count > 1;
```

	title	genre_count
▶	Space Quest	2

-- =====

-- 25. Get users whose subscription is expiring next week

-- =====

```
SELECT u.email, s.next_billing_at
FROM subscriptions s
JOIN users u ON u.id = s.user_id
WHERE s.next_billing_at BETWEEN NOW() AND NOW() + INTERVAL 7 DAY;
```

	email	next_billing_at
	carol@example.com	2025-11-19 14:07:21